



TCC805x MCU BSP

API Specification

for Inter Processor Communication

Rev. 1.00 [A]

2020-11-12

※ The information in this document is subject to change without notice and should not be construed as a commitment by Telechips Inc.

Kindly visit <http://www.telechips.com> for more information.

TABLE OF CONTENTS

Contents

1	Introduction	3
2	Terms and Abbreviations	3
3	Source Files.....	4
3.1	Location of Source Files.....	4
3.2	Simple Description of Source Files	4
4	Enumerations	6
4.1	IPCSvcCh_t.....	6
4.2	IPCIoCmd_t	7
5	Constants.....	8
5.1	IPC Error Messages.....	8
5.2	Sample IPC Command Defines.....	9
6	Callback Functions	10
6.1	IPCCallback.....	10
7	APIs	11
7.1	I2C_Init.....	11
7.2	IPC_RegisterCbFunc.....	12
7.3	IPC_SendPacket	13
8	How to use IPC Application Driver	14
8.1	Handling Received Data	14
8.2	Sending Data	14
9	References	15
10	Revision History	16
	Rev. 1.00: 2020-11-12	16

Figures

Figure 3.1 Location of Source Files	4
---	---

Tables

Table 2.1 Terms and Abbreviation	3
--	---

1 INTRODUCTION

This document describes IPC application driver of TCC805x MCU BSP. The IPC application driver provides a function to communicate with program running on other cores such as Cortex-A53 and Cortex-A7. The IPC application driver internally uses mailbox hardware for data transfer among the application processors. Therefore, it is mandatory for Coretex-A53 and Cortex-A7 to use mailbox and IPC application driver provided by Telechips.

2 TERMS AND ABBREVIATIONS

Table 2.1 Terms and Abbreviation

MCU	Micro Controller Unit
RTOS	Real Time Operating System
BSP	Board Support Package
API	Application Programmable Instruction
FIFO	First In First Out
IPC	Inter-Processor Communication (Using mailbox as a physical channel)

3 SOURCE FILES

3.1 Location of Source Files

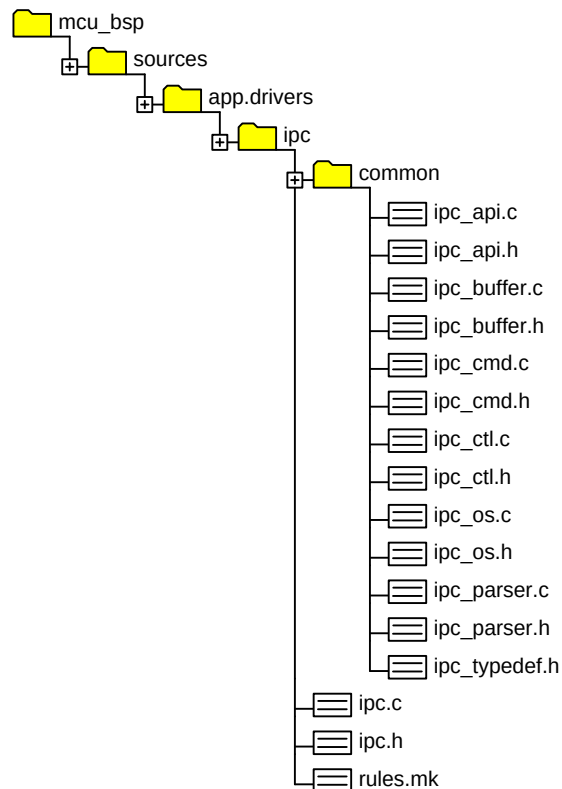


Figure 3.1 Location of Source Files

3.2 Simple Description of Source Files

- `ipc_api.c`
 - Source code for internal APIs and common software implementations which is not related to hardware
- `ipc_api.h`
 - Header file for pre-defined values to control IPC packets
- `ipc_buffer.c`
 - Source code for IPC read buffer
- `ipc_buffer.h`
 - Header file for IPC read buffer
- `ipc_cmd.c`
 - Source code for IPC protocol command
- `ipc_cmd.h`
 - Header file for IPC protocol command
- `ipc_ctl.c`
 - Source code for IPC control task for reading

- ipc_ctl.h
 - Header file for IPC control task for reading
- ipc_os.c
 - Source code for using Operating System resources (System Adaptation Layer)
- ipc_os.h
 - Header file for using Operating System resources (System Adaptation Layer)
- ipc_parser.c
 - Source code for IPC protocol parser.
- ipc_parser.h
 - Header file for IPC protocol parser
- ipc_typedef.h
 - Header file for IPC command types
- ipc.c
 - Source code for user APIs
- ipc.h
 - Header file for user APIs
- rules.mk
 - Build options and target information for IPC application driver

4 ENUMERATIONS

4.1 IPCSvcCh_t

Enumeration variables for IPC communication channels. Only non-secure channels are available for default released package. And secure channels can be used if NS bit is reset in **SCR_EL3** register of Cortex-R5 CPU. **SCR_EL3** register can be accessed on bootloader.

Declaration

```
typedef enum __IPCSvcCh__
{
    MBOX_SVC_CH_CA53_SECURE,
    MBOX_SVC_CH_CA53_NONSECURE,
    MBOX_SVC_CH_CA72_SECURE,
    MBOX_SVC_CH_CA72_NONSECURE,
    MBOX_SVC_CH_MAX,
} MBOXSvcCh_t;
```

Description

Value	Description
MBOX_SVC_CH_CA53_SECURE	For communication with CA53 secure Mailbox
MBOX_SVC_CH_CA53_NONSECURE	For communication with CA53 non-secure Mailbox
MBOX_SVC_CH_CA72_SECURE	For communication with CA72 secure Mailbox
MBOX_SVC_CH_CA72_NONSECURE	For communication with CA72 non-secure Mailbox
MBOX_SVC_CH_MAX	For checking validation

Remark

None

4.2 IPCIoCmd_t

Enumeration variables for I/O command of *IPC_ioctl* function. This enumeration is used at internal IPC application driver code.

Declaration

```
typedef enum _IPCIoCmd_  
{  
    IOCTL_IPC_WRITE      = 0,  
    IOCTL_IPC_READ       = 1,  
    IOCTL_IPC_SET_PARAM  = 2,  
    IOCTL_IPC_GET_PARAM  = 3,  
    IOCTL_IPC_PING_TEST  = 4,  
    IOCTL_IPC_FLUSH      = 5,  
    IOCTL_IPC_ISREADY    = 6,  
} IPCIoCmd_t;
```

Description

Value	Description
IOCTL_IPC_WRITE	Send command and data to application processor.
IOCTL_IPC_READ	Receive command and data from application processor.
IOCTL_IPC_SET_PARAM	Set blocking read timeout.
IOCTL_IPC_GET_PARAM	Get blocking read timeout.
IOCTL_IPC_PING_TEST	Send ping message to application processor and waits ACK.
IOCTL_IPC_FLUSH	Deprecated
IOCTL_IPC_ISREADY	Check that IPC channel is opened at application processor.

Remark

None

5 CONSTANTS

5.1 IPC Error Messages

These error messages are caused by implementation of IPC application driver. These errors mean that there are some problems to use IPC application driver.

Declaration

```
#define IPC_SUCCESS          (0)
#define IPC_ERR_COMMON      (-1)
#define IPC_ERR_BUSY        (-2)
#define IPC_ERR_NOTREADY    (-3)
#define IPC_ERR_TIMEOUT     (-4)
#define IPC_ERR_WRITE        (-5)
#define IPC_ERR_READ        (-6)
#define IPC_ERR_BUFFER       (-7)
#define IPC_ERR_ARGUMENT     (-8)
#define IPC_ERR_RECEIVER_NOT_SET (-9)
#define IPC_ERR_RECEIVER_DOWN (-10)
#define IPC_ERR_INVALID_DL   (-800)
#define IPC_ERR_NOT_SUPPORTED (-900)
```

Description

Value	Description
IPC_SUCCESS	On success
IPC_ERR_COMMON	IPC common error
IPC_ERR_BUSY	IPC application driver is already opened.
IPC_ERR_NOTREADY	IPC application driver is not initialized.
IPC_ERR_TIMEOUT	IPC Blocking read timeout
IPC_ERR_WRITE	IPC write error
IPC_ERR_READ	IPC read error
IPC_ERR_BUFFER	IPC put/get buffer error
IPC_ERR_ARGUMENT	IPC invalid argument
IPC_ERR_RECEIVER_NOT_SET	Reserved
IPC_ERR_RECEIVER_DOWN	IPC send ACK error

Remark

None

5.2 Sample IPC Command Defines

These definitions are used only for sample applications using IPC communication. The applications using IPC communication are located in sources/app.sample/app.key and sources/app.sample/app.system.monitoring.

Declaration

```
#define TCC_IPC_CMD_KEY          (0x02)
#define TCC_IPC_CMD_KEY_EXT      (0x01)
#define TCC_IPC_CMD_KEY_ROTARY   (0x02)
#define TCC_IPC_CMD_SYS_MONITORING (0x04)
#define IPC_SYS_MON_RECOG_REQ     (0x01) //(MICOM => Peer Core)
#define IPC_SYS_MON_RECOG_RES     (0x01) //(Peer Core => MICOM)
```

Description

Value	Description
TCC_IPC_CMD_KEY	Command ID for app.key
TCC_IPC_CMD_KEY_EXT	External Key ID for app.key
TCC_IPC_CMD_KEY_ROTARY	Rotary Key ID for app.key
TCC_IPC_CMD_SYS_MONITORING	Command ID for app.system.monitoring
IPC_SYS_MON_RECOG_REQ	Request ID for app.system.monitoring
IPC_SYS_MON_RECOG_RES	Response ID for app.system.monitoring

Remark

None

6 CALLBACK FUNCTIONS

6.1 IPCCallback

This type is defined for a callback function to receive IPC data at user application. The IPC control task receives IPC data from mailbox driver and forwards it to parameters of this callback function.

Declaration

```
typedef void (*IPCCallback)
(
    uint16 uiCmd,
    uint8 *pucData,
    uint16 uiLength
);
```

Parameters

Value	Description
<i>uiCmd</i>	<i>2 bytes of received command data</i>
<i>pucData</i>	<i>A pointer of received payload data</i>
<i>uiLength</i>	<i>Length of received payload data</i>

Return

None

Remark

None

7 APIs

7.1 I2C_Init

Initializes the IPC application driver. It creates IPC control tasks and opens IPC application drivers for A72 non-secure channel and A53 non-secure channel. Negotiation messages are sent when IPC application driver is opened.

Declaration

```
void IPC_Create  
(  
    void  
);
```

Parameters

None

Return

None

Remark

None

7.2 IPC_RegisterCbFunc

Registers callback function for receiving data with specific channel and application identifier. IPC data must be received by using callback function. It is not allowed to receive data by using IPC_loctl.

Declaration

```
void IPC_RegisterCbFunc
(
    IPCSvcCh_t    siCh,
    uint8         ucId,
    IPCCallback   func,
    void *        pArg1,
    void *        pArg2
);
```

Parameters

Value	Description
siCh	Channel number for IPC service (IPCSvcCh_t) - IPC_SVC_CH_MAIN for Coretax-A53 non-secure channel - IPC_SVC_CH_SUB for Coretax-A7 non-secure channel
ucId	An application identifier (Must be the same with the application identifier of application processor)
func	A callback function pointer. Refer to the IPCCallback . - uiCmd: User defined command that is received - pucData: User defined data that is received - uiLength: Length of received pucData (Maximum 512 bytes)
pArg1	NULL (Reserved)
pArg2	NULL (Reserved)

Return

None

Remark

None

7.3 IPC_SendPacket

Sends user defined 2 bytes commands and 512 bytes data to application program running on application processor.

Declaration

```
int32 IPC_SendPacket
(
    IPCSvcCh_t    siCh,
    uint16        uiCmd1,
    uint16        uiCmd2,
    const uint8 * pucData,
    uint16        uiLength
);
```

Parameters

Value	Description
<i>siCh</i>	An IPC service channel to be initialized. It must be one of <i>IPCSvcCh_t</i> . Refer to the IPCSvcCh_t .
<i>uiCmd1</i>	An application identifier
<i>uiCmd2</i>	A user defined command to be sent
<i>pucData</i>	A user defined data within 512 bytes
<i>uiLength</i>	A length of <i>pucData</i> to be sent

Return

Value	Description
<i>IPC_SUCCESS</i>	Success
<i>IPC_ERR_NOTREADY</i>	IPC application driver does not complete negotiation (handshake) with application processor.
<i>IPC_ERR_COMMON</i>	IPC application driver internal error. Need to debug the inside of IPC application driver.
<i>IPC_ERR_ARGUMENT</i>	Invalid parameter is passed.

Remark

None

8 HOW TO USE IPC APPLICATION DRIVER

8.1 Handling Received Data

```
IPC_RegisterCbFunc(IPC_SVC_CH_CA53_NONSECURE, TCC_IPC_CMD_SYS_MONITORING, (IPCCallback)SMA_IpcCbFunc,
NULL, NULL);

static void SMA_IpcCbFunc(uint16 uhwCmd, uint8 *pucData, uint16 uhwLength)
{
    printf("received command %d\n", uhwCmd);
    printf("received data [");

    for (index = 0; index < uhwLength; index++)
        printf("0x%02X ", pucData[index]);

    printf("\n");
}
```

8.2 Sending Data

```
static void KEY_AppAdcTask(void * pArgs)
{
    ...

    while (1)
    {
        ret_key = KEY_AdcScan(&ext_key, &key_status);

        if(ret_key == SAL_RET_SUCCESS)
        {
            ...
            ipc_ret = IPC_SendPacket(IPC_SVC_CH_CA53_NONSECURE, (uint16)TCC_IPC_CMD_KEY, (uint16)
                TCC_IPC_CMD_KEY_EXT, (uint8*)keyData, (uint16)2);
            ...
            printf("%s, [ADC_KEY_IPC_SEND] ext_key(keyData[0]) : 0x%x, key_status(keyData[1]) : 0x%x,
                size : 2, ret : %d\n", __func__, keyData[0], keyData[1], ipc_ret);
        }

        SAL_TaskSleep(10);
    }
}
```

9 REFERENCES

- [1] TCC805x Full Specification
- [2] Contact Telechips for more details: sales@telechips.com

10 REVISION HISTORY

Rev. 1.00: 2020-11-12

- First version release

DISCLAIMER

All information and data contained in this material are without any commitment, are not to be considered as an offer for conclusion of a contract, nor shall they be construed as to create any liability. Any new issue of this material invalidates previous issues. Product availability and delivery are exclusively subject to our respective order confirmation form; the same applies to orders based on development samples delivered. By this publication, Telechips, Inc. does not assume responsibility for patent infringements or other rights of third parties that may result from its use.

Further, Telechips, Inc. reserves the right to revise this publication and to make changes to its content, at any time, without obligation to notify any person or entity of such revisions or changes.

No part of this publication may be reproduced, photocopied, stored on a retrieval system, or transmitted without the express written consent of Telechips, Inc.

This product is designed for general purpose, and accordingly customer be responsible for all or any of intellectual property licenses required for actual application. Telechips, Inc. does not provide any indemnification for any intellectual properties owned by third party.

Telechips, Inc. can not ensure that this application is the proper and sufficient one for any other purposes but the one explicitly expressed herein. Telechips, Inc. is not responsible for any special, indirect, incidental or consequential damage or loss whatsoever resulting from the use of this application for other purposes.

COPYRIGHT STATEMENT

Copyright in the material provided by Telechips, Inc. is owned by Telechips unless otherwise noted.

For reproduction or use of Telechips' copyright material, permission should be sought from Telechips. That permission, if given, will be subject to conditions that Telechips' name should be included and interest in the material should be acknowledged when the material is reproduced or quoted, either in whole or in part. You must not copy, adapt, publish, distribute or commercialize any contents contained in the material in any manner without the written permission of Telechips. Trade marks used in Telechips' copyright material are the property of Telechips.

Important Notice

This material may include technology owned by the 3rd party licensor and the technology may be subject to its associated licenses. It is solely customer's responsibility to identify and comply with such licenses. No other licenses are granted or implied by Telechips with making available this material.

For customers who use licensed Codec ICs and/or licensed codec firmware of mp3:

"Supply of this product does not convey a license nor imply any right to distribute content created with this product in revenue-generating broadcast systems (terrestrial. Satellite, cable and/or other distribution channels), streaming applications(via internet, intranets and/or other networks), other content distribution systems(pay-audio or audio-on-demand applications and the like) or on physical media(compact discs, digital versatile discs, semiconductor chips, hard drives, memory cards and the like). An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use other firmware of mp3:

"Supply of this product does not convey a license under the relevant intellectual property of Thomson and/or Fraunhofer Gesellschaft nor imply any right to use this product in any finished end user or ready-to-use final product. An independent license for such use is required. For details, please visit <http://mp3licensing.com>".

For customers who use Digital Wave DRA solution:

"Supply of this implementation of DRA technology does not convey a license nor imply any right to this implementation in any finished end-user or ready-to-use terminal product. An independent license for such use is required."

For customers who use DTS technology:

"This product made under license to certain U.S. patents and/or foreign counterparts."

"© 1996 – 2011 DTS, Inc. All rights reserved."

For customers who use Dolby technology:

"Supply of this Implementation of Dolby technology does not convey a license nor imply a right under any patent, or any other industrial or intellectual property right of Dolby Laboratories, to use this Implementation in any finished end-user or ready-to-use final product. It is hereby notified that a license for such use is required from Dolby Laboratories."

For customers who use Google technology:

"Copyright © 2013 Google Inc. All rights reserved."

For customers who use MS technology:

"This product is subject to certain intellectual property rights of Microsoft and cannot be used or distributed further without the appropriate license(s) from Microsoft."